

مستند فنی پیاده‌سازی سرویس Topup در معماری پرداخت

مقدمه

در این پروژه قابلیت شارژ فوری (Topup) به ساختار فعلی سیستم پرداخت اضافه شده است. هدف طراحی، پیاده‌سازی یک جریان تراکنشی قابل توسعه، مقاوم در برابر خطا و نزدیک به معماری Production در سیستم‌های بانکی و PSP بوده است.

در ساختار موجود، Switch مسئول مدیریت تراکنش‌های مالی است و تراکنش‌ها مطابق استانداردهای پرداخت شامل مراحل زیر هستند:

1. Purchase Transaction

2. Advice (Confirmation)

3. Reverse Transaction

در این سناریو، سرویس جدید Topup به سیستم اضافه شده است تا پس از موفقیت Purchase، عملیات شارژ سیم‌کارت از طریق سرویس MCI انجام شود.

معماری کلی سیستم

معماری سیستم بر اساس الگوی Microservice و Event-Driven طراحی شده است.

اجزای اصلی سیستم:

- Switch API
- Outbox Pattern
- InMemory Queue
- Topup Worker
- MCI Client
- Saga Orchestrator

جریان کامل تراکنش

مرحله 1 : دریافت درخواست Purchase

کاربر از طریق Terminal درخواست شارژ ارسال می‌کند.

Switch درخواست را دریافت کرده و:

- تراکنش را در دیتابیس ذخیره می‌کند
 - وضعیت اولیه تراکنش را PendingPurchase قرار می‌دهد
 - یک Event در جدول Outbox ذخیره می‌کند
- هدف از Outbox جلوگیری از از دست رفتن Event در صورت Failure است.

مرحله 2 : Publish Event

Background Worker مربوط به Outbox به صورت دوره‌ای پیام‌های پردازش نشده را از جدول Outbox خوانده و داخل Queue منتشر می‌کند.

این کار باعث می‌شود:

- ارسال پیام قابل اطمینان باشد
- وابستگی مستقیم بین Switch و Topup حذف شود
- سیستم Async و Decoupled شود

مرحله 3 : Consume Event توسط TopupWorker

TopupWorker پیام را از Queue دریافت می‌کند.

در این مرحله:

- Saga Orchestrator شروع می‌شود
- وضعیت تراکنش به TopupProcessing تغییر می‌کند

Saga Pattern

برای مدیریت صحیح فرآیند مالی از Saga Pattern استفاده شده است.

هدف: Saga

مدیریت یک Business Transaction شامل چند مرحله مستقل بدون استفاده از Distributed Transaction است.

در این سناریو مراحل Saga شامل:

1. Purchase

2. Topup

3. Advice

و در صورت: Failure

Reverse 4.

می‌باشد.

نقش Saga Orchestrator

Saga Orchestrator مسئول کنترل State فرآیند است.

وظایف:

- مدیریت مراحل تراکنش
- ذخیره وضعیت Saga
- تصمیم‌گیری در Success / Failure
- اجرای Compensation Transaction

State های Saga :

- Started
- PurchaseCompleted
- TopupCompleted
- AdviceCompleted
- ReverseCompleted

ارتباط با MCI

TopupWorker از طریق MciClient با سرویس MCI ارتباط برقرار می‌کند.

در این بخش شماره موبایل و مبلغ شارژ به MCI ارسال می‌شود.

Polly با Retry Mechanism

برای جلوگیری از Failure موقت سرویس خارجی از Polly استفاده شده است.

در صورت Failure :

- درخواست چند بار Retry می‌شود

- Retry ها به صورت Exponential Backoff انجام می‌شوند

نمونه:

- Retry اول → 2 ثانیه

- Retry دوم → 4 ثانیه

- Retry سوم → 8 ثانیه

این طراحی باعث افزایش Resiliency سیستم می‌شود.

Circuit Breaker

برای جلوگیری از Cascading Failure از Circuit Breaker استفاده شده است.

اگر تعداد خطاهای MCI از حد مشخصی بیشتر شود:

- Circuit باز می‌شود

- درخواست جدید ارسال نمی‌شود

- سیستم Fail-Fast عمل می‌کند

مزایا:

- محافظت از سیستم

- جلوگیری از Overload

- کاهش فشار روی Dependency خارجی

Advice Flow

اگر عملیات Topup موفق باشد:

- Saga وارد مرحله TopupCompleted می‌شود

- سرویس Advice توسط Switch اجرا می‌شود

- وضعیت نهایی تراکنش Completed خواهد شد

Advice معادل Confirmation در سیستم پرداخت است.

Reverse Flow

اگر پس از Retry ها همچنان عملیات Topup ناموفق باشد:

- Saga عملیات Compensation را اجرا می‌کند
 - سرویس Reverse توسط Switch فراخوانی می‌شود
 - مبلغ به کاربر بازگردانده می‌شود
 - وضعیت تراکنش Reversed خواهد شد
- Reverse به عنوان Compensating Transaction در Saga عمل می‌کند.

Idempotency

برای جلوگیری از تکرار تراکنش تکراری از Idempotency Key استفاده شده است.

در صورتی که درخواست تکراری ارسال شود:

- تراکنش جدید ایجاد نمی‌شود
- نتیجه قبلی بازگردانده می‌شود

دلیل استفاده از Async Processing

ارتباط مستقیم و Sync بین MCI و Switch می‌تواند باعث:

- Block شدن سیستم
- کاهش Throughput
- افزایش Failure Propagation

شود.

به همین دلیل از Queue و Async Processing استفاده شده است.

مزایا:

- Scalability
- Decoupling
- Fault Tolerance
- High Availability

دلیل استفاده از Outbox Pattern

اگر پس از ذخیره تراکنش، سیستم قبل از Publish پیام Crash کند، Event از بین می‌رود.

Outbox این مشکل را حل می‌کند.

در این روش:

- Transaction

- Event

در یک Persistence Boundary ذخیره می‌شوند.

ساختار نهایی سیستم

جریان نهایی:

Purchase

→ Outbox

→ Queue

→ Saga Orchestrator

→ MCI Retry + Circuit Breaker

→ Success → Advice

→ Failure → Reverse

نکات معماری

این طراحی بر اساس اصول زیر انجام شده است:

- Event-Driven Architecture

- Eventual Consistency

- Saga Pattern

- Retry & Resiliency

- Decoupling

- Compensating Transaction

- High Availability

محدودیت‌های نسخه فعلی

به دلیل Challenge بودن پروژه:

- Queue به صورت InMemory پیاده‌سازی شده است
- Database از نوع InMemoryDatabase است
- MCI به صورت Mock شبیه‌سازی شده است

در محیط واقعی می‌توان از:

- Kafka / RabbitMQ
- SQL Server / PostgreSQL
- OpenTelemetry
- Distributed Tracing

استفاده کرد.

جمع‌بندی

در این پروژه تلاش شده است حتی با وجود استفاده از ابزارهای ساده InMemory ، معماری سیستم نزدیک به Production طراحی شود و تمامی Concern های مهم سیستم‌های مالی شامل:

- Reliability
- Idempotency
- Resiliency
- Compensation
- Scalability

در نظر گرفته شوند.